208. Implement Trie (Prefix Tree) Medium

Acceptance: 69.0% | LeetCode

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string `word` into the trie.
- `boolean search(String word)` Returns `true` if the string `word` is in the trie (i.e., was inserted before), and `false` otherwise.
- `boolean startsWith(String prefix)` Returns `true` if there is a previously inserted string `word` that has the prefix `prefix`, and `false` otherwise.

Example 1:

```
**Input**
["Trie", "insert", "search", "search", "startsWith",
"insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"],
["app"]]
**Output**
[null, null, true, false, true, null, true]

**Explanation**
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple");   // return True
trie.search("app");     // return False
trie.startsWith("app"); // return True
trie.insert("app");
trie.search("app");     // return True
```

Constraints:

- `1 <= word.length, prefix.length <= 2000`
- `word` and `prefix` consist only of lowercase English letters.
- At most `3 * 104` calls **in total** will be made to `insert`, `search`, and `startsWith`.

Claude Opus Solution

Implement Trie (Prefix Tree)

1. Intuition

The key insight is recognizing that strings often **share common prefixes**. Instead of storing each word separately, we can store characters in a **tree structure** where each path from root to a node represents a prefix.

```
Inserting "apple" and "app":

         root
          |
          a
          |
          p
          |
          p  ← marked as END (word "app")
          |
          l
          |
          e  ← marked as END (word "apple")
```

This sharing lets us: - **Search/Insert in O(word length)** — independent of how many words exist - **Efficiently check prefixes** — the foundational structure of the trie

The critical distinction: **`search` requires hitting an "end-of-word" marker**, while **`startsWith` just needs the path to exist**. This is why we need to mark which nodes terminate a complete word.

---

2. Approach (Pseudocode)

```
INIT:
    root = empty dictionary

INSERT(word):
    node = root
    for each char in word:
        if char not in node:
            create empty dict at node[char]
```

```
        node = node[char]      # descend into the tree
    mark node as end-of-word   # use special key like "#"

SEARCH(word):
    node = traverse(word)
    return node exists AND node is marked end-of-word

STARTS_WITH(prefix):
    node = traverse(prefix)
    return node exists   # path existing is enough

TRAVERSE(s):              # helper to walk down the trie
    node = root
    for each char in s:
        if char not in node: return None  # path broken
        node = node[char]
    return node
```

---

3. Python Solution

```python
class Trie:
    def __init__(self):
        # The trie is represented as nested dictionaries.
        # Each dict maps a character -> child dict.
        # A special key "#" marks the end of a complete word.
        self.root = {}

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            # If this character path doesn't exist yet, create it.
            # setdefault returns the existing child or creates a new empty dict.
            node = node.setdefault(char, {})
        # Mark the final node as the end of a valid word.
        # We use "#" because it can't collide with lowercase letters.
        node["#"] = True

    def _traverse(self, s: str):
        """Walk down the trie following s; return the final node or None."""
        node = self.root
        for char in s:
            if char not in node:
                return None  # Path breaks → prefix/word doesn't exist
            node = node[char]
        return node

    def search(self, word: str) -> bool:
        node = self._traverse(word)
        # Word exists ONLY if path exists AND it was explicitly inserted
        # (i.e., the end-of-word marker "#" is present).
        return node is not None and "#" in node

    def startsWith(self, prefix: str) -> bool:
        # For a prefix, we only care that the path exists.
        # No need to check for the end-of-word marker.
        return self._traverse(prefix) is not None
```

Why the `"#"` marker matters

Consider inserting only `"apple"`, then calling `search("app")`: - The path `a→p→p` **exists** (it's part of "apple") - But `"app"` was never explicitly inserted, so there's **no `"#"` marker** at that node - `search` correctly returns `False`, while `startsWith` returns `True` ✓

---

4. Complexity Analysis

Let **L** = length of the word/prefix, **N** = total characters inserted.

Operation	Time	Space
`insert`	O(L)	O(L) worst case (all new nodes)

Operation	Time	Space
`search`	O(L)	O(1)
`startsWith`	O(L)	O(1)

- **Time**: Each operation processes the string character by character. Dictionary lookups are O(1) on average, so the cost is proportional to the string length **L**, independent of the number of stored words.
- **Space**: Total trie space is **O(N)** in the worst case (no shared prefixes). With heavy prefix sharing, space is much smaller — this is the trie's main advantage over storing words in a set.

---

5. Alternative Approaches

A. TrieNode Class (more explicit / OOP)

Instead of nested dicts, use an explicit node class. Cleaner for adding features later (e.g., counting words).

```python
class TrieNode:
    def __init__(self):
        self.children = {}      # char -> TrieNode
        self.is_end = False     # explicit end-of-word flag

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True

    def _traverse(self, s):
        node = self.root
        for char in s:
            if char not in node.children:
                return None
            node = node.children[char]
        return node

    def search(self, word: str) -> bool:
        node = self._traverse(word)
        return node is not None and node.is_end

    def startsWith(self, prefix: str) -> bool:
        return self._traverse(prefix) is not None
```

B. Fixed-size array of children

Since input is only lowercase letters (`'a'`–`'z'`), each node can hold a 26-element array indexed by `ord(char) - ord('a')`. This trades extra memory per node for slightly faster, hash-free access — useful in performance-critical languages, but in Python dicts are usually preferred for clarity and memory efficiency on sparse data.

C. HashSet of all words/prefixes (naive)

Store every word in a set, plus every prefix in another set. This makes lookups O(L) too, but **wastes huge space** (storing all prefixes of all words) and loses the trie's elegance. Not recommended.

Pattern takeaway: The Trie is the go-to structure whenever you need **prefix-based operations** — autocomplete, spell-checking, word search puzzles (LeetCode 212), and IP routing. The nested-dictionary approach (the reference solution) is the most Pythonic and concise way to implement it.

Codex Solution

1. Intuition

A **Trie** stores strings character by character in a tree-like structure.

The key idea:

- Each node represents a prefix.
- Each edge represents a character.
- Words that share prefixes also share the same path in the trie.

For example, inserting `"apple"` and `"app"`:

```
root
 └── a
     └── p
         └── p   ← end of "app"
             └── l
                 └── e   ← end of "apple"
```

We need a way to distinguish:

- `"app"` is just a prefix of `"apple"`
- `"app"` is also an actual inserted word

So each node stores whether a word ends there.

---

2. Approach

We can represent each Trie node as:

- A dictionary mapping characters to child nodes
- A boolean flag `is_word` indicating whether a word ends at this node

Insert

To insert a word:

```
start at root

for each character in word:
    if character does not exist as a child:
        create a new node
    move to that child node

mark the final node as the end of a word
```

Search

To search for a full word:

```
start at root

for each character in word:
    if character does not exist:
        return False
    move to that child node

return True only if current node marks the end of a word
```

StartsWith

To check if a prefix exists:

```
start at root

for each character in prefix:
    if character does not exist:
        return False
    move to that child node

return True
```

The difference between `search` and `startsWith` is:

- `search("app")` requires `"app"` to be a complete inserted word
- `startsWith("app")` only requires that some inserted word begins with `"app"`

---

3. Python Solution

```python
class TrieNode:
    def __init__(self):
        # Maps a character to the next TrieNode
        # Example: children['a'] gives the node after character 'a'
        self.children = {}

        # True if a complete word ends at this node
        self.is_word = False


class Trie:

    def __init__(self):
        # Root node does not represent any character.
        # It is just the starting point of all words.
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        # Start from the root node
        current = self.root

        # Traverse each character in the word
        for char in word:
            # If the character path does not exist, create it
            if char not in current.children:
                current.children[char] = TrieNode()

            # Move to the child node for this character
            current = current.children[char]

        # After inserting all characters, mark this node as a word ending
        current.is_word = True

    def search(self, word: str) -> bool:
        # Start from the root node
        current = self.root

        # Traverse the trie according to the word characters
        for char in word:
            # If any character path is missing, the word does not exist
            if char not in current.children:
                return False

            # Move to the next node
            current = current.children[char]

        # The path exists, but we must ensure this exact word was inserted
        return current.is_word

    def startsWith(self, prefix: str) -> bool:
        # Start from the root node
        current = self.root

        # Traverse the trie according to the prefix characters
        for char in prefix:
            # If any character path is missing, no inserted word has this prefix
            if char not in current.children:
                return False

            # Move to the next node
            current = current.children[char]

        # If we successfully traversed the entire prefix, it exists
        return True
```

4. Complexity Analysis

Let:

- `L` be the length of the input `word` or `prefix`

`insert(word)`

Each character is processed once.

```
Time Complexity: O(L)
```

In the worst case, every character creates a new Trie node.

```
Space Complexity: O(L)
```

`search(word)`

Each character is checked once.

```
Time Complexity: O(L)
```

No new nodes are created.

```
Space Complexity: O(1)
```

`startsWith(prefix)`

Each character is checked once.

```
Time Complexity: O(L)
```

No new nodes are created.

```
Space Complexity: O(1)
```

Overall Space Complexity

If many words are inserted, the total space depends on the number of unique prefixes.

In the worst case, no words share prefixes.

If the total number of inserted characters is `N`:

```
Overall Space Complexity: O(N)
```

---

5. Alternative Approaches

1. Nested Dictionary Trie

Instead of defining a `TrieNode` class, we can use nested dictionaries directly.

Example idea:

```
{
    "a": {
        "p": {
            "p": {
                "#": True
            }
        }
    }
}
```

A special marker like `"#"` or `"."` indicates the end of a word.

This is shorter, but slightly less explicit.

---

2. Array-Based Trie

Because the problem only uses lowercase English letters, each node can store an array of size `26`.

```
children = [None] * 26
```

Then the index for each character is:

```
index = ord(char) - ord('a')
```

This can be faster than dictionaries, but it may use more memory because every node stores space for all 26 possible children.

---

3. Hash Set of Words and Prefixes

We could store:

- All complete words in one set
- All prefixes in another set

For example, inserting `"apple"` would store:

```
words = {"apple"}
prefixes = {"a", "ap", "app", "appl", "apple"}
```

Then:

- `search(word)` checks `word in words`
- `startsWith(prefix)` checks `prefix in prefixes`

This gives very fast lookups, but insertion requires storing every prefix separately, which can use more memory than a trie.